

COSC2446 Web Programming

AD006P24 - Associate Degree in Information Technology

COSC2446 Web Programming A

Assessment Number: 2 Dynamic Website & Demonstration Assignment			
Weighting	30%	Due Date	1 st of May 2026, COB (5:30 pm)
Course coordinator	Tanya Unterberger	Contact details	tanya.unterberger@rmit.edu.au (for general enquires only)
Individual or Group Assessment	Individual		

This assessment task is expected to take you a minimum of 40-50 hours to complete.

Task

This assessment includes a practical submission and a scheduled one-on-one demonstration.

Students submit a complete, working solution (front-end or full-stack website) based on the requirements outlined in the task. They are permitted to use AI tools to assist with development, provided all AI-generated content is appropriately acknowledged.

In the demonstration, you will walk through your submitted site, explain how key parts of your code work, and answer a few short questions related to the technologies used. You are allowed to use AI tools in your work, but you are expected to understand and be able to explain any code submitted.

Understanding Your Work

During your scheduled demonstration, you will be required to explain how key parts of your submitted code work. This is a vital component of the assessment.

If you are unable to demonstrate a clear understanding of your own code — regardless of whether it was written by you or with the assistance of AI tools — you will not be eligible to receive more than a **Pass (50%)** for the task.

This policy supports academic integrity and ensures that all students meet the learning outcomes of the course.

What is being assessed

In this assessment, the following course outcomes are being assessed:

1. Define the protocols and systems used on the Web

COSC2446 Web Programming

2. Explain the functions of clients and servers on the Web.
3. Create standards-compliant web pages.
4. Implement client-side and server-side scripting.
5. Design and implement an interactive web site with regard to issues of usability and accessibility.

The following program outcomes are incorporated into this assessment task:

1. Enabling Knowledge
2. Critical Analysis
3. Problem Solving
4. Communication
5. Responsibility

Assessment details

This assignment aims to further develop PetConnect, a pet adoption platform using PHP (procedural with MySQLi), MySQL, and Bootstrap 5. The application will be a dynamic website, and the content database-driven.

The submission must demonstrate secure server-side development, client-side validation, reusable components, and adherence to best practices in code structure and deployment. The project must be completed and submitted individually.

This assessment includes a practical submission and a scheduled one-on-one demonstration.

AI TOOLS AND THEIR USE ARE NOT RESTRICTED FOR THIS ASSESSMENT TASK

Presentation requirements

In the demonstration, you will walk through your submitted site, explain how key parts of your code work, and answer a few short questions related to the technologies used.

Each student attends a scheduled session where they are asked to:

- Demonstrate the functionality of their submission.
- Explain key parts of their own code to verify their understanding.
- Answer short questions related to fundamental concepts covered in the assessment (HTML semantics, CSS layout, JavaScript logic, database queries, etc.).

COS2446 Web Programming

The demonstration is not only to show functionality but to verify authorship and comprehension, supporting academic integrity in a context where AI use is permitted.

Assessment criteria

You will be assessed on the following criteria:

Website Structure and Design	GitHub and Deployment
Database Structure and Design	Documentation and Submission
Technical Implementation	Adherence to requirements
Secure coding	Usability and validity

Refer to the assessment rubric for more details on grading.

Assessment 2 Checklist

#	Area	Requirement	Completed (Y/N)
Stage 1 – File Structure, Includes & Layout			
1	Structure	All required PHP files exist in the a2 directory	
2	Structure	Required folders exist (assets, includes, assets/images/pets)	
3	Includes	includes/db_connect.inc exists and supports localhost & Jacob 5	
4	Includes	includes/header.inc present and included on all pages	
5	Includes	includes/nav.inc present and included on all pages	
6	Includes	includes/footer.inc present and included on all pages	
7	Metadata	All pages have unique and meaningful <title> tags	
8	Navigation	All pages have consistent navigation links	
9	Navigation	Navbar includes PetConnect logo linking to home	
10	Navigation	Navbar is responsive	
11	Layout	Semantic HTML used (header, nav, main, footer)	
12	Layout	Responsive design implemented with Bootstrap 5	
13	Layout	Header present on all pages	
14	Layout	Navigation present on all pages	
15	Layout	Main content area present on all pages	
16	Layout	Footer present on all pages. Includes your name	
Stage 2 – Database & Dynamic Content			
17	Database	Database created on Jacob 5 using correct schema	
18	Database	Table and column names match provided schema exactly	
19	Database	All DB access uses MySQLi procedural prepared statements	
20	Database	No SQL injection vulnerabilities	
21	Home Page	index.php displays carousel of static images	
22	Home Page	index.php displays grid of latest 4 pets from database	
23	Pets Page	pets.php uses two-column layout	
24	Pets Page	pets.php lists records from database	
25	Pets Page	Pet names link to details.php	
26	Details Page	details.php displays a single record via query string	

COSC2446 Web Programming

27	Gallery	gallery.php displays grid of images from database
28	Gallery	Each gallery image links to corresponding record
29	Add Page	add.php contains functional form for adding pets
30	Add Page	Form submission adds record to database
31	Add Page	Uploaded image saved to assets/images/pets/
32	Uploads	Uploaded images use server-generated unique filenames (e.g. uniqid() or timestamp-based naming)
Stage 3 – JavaScript, CSS & Interactivity		
33	JavaScript	Single custom JS file used (assets/js/scripts.js)
34	JavaScript	No inline JavaScript used
35	JavaScript	Image extension validation implemented in JS
36	JavaScript	Only jpg, jpeg, png, gif, webp accepted
37	JavaScript	Bootstrap modal implemented for gallery images. Modal opens and closes correctly
38	JavaScript	Pets can be filtered by status on gallery page
39	CSS	Single custom CSS file used (assets/css/styles.css)
40	CSS	No inline styles used
41	Fonts	Inter font used for body text
42	Fonts	Poppins font used for headings
43	Icons	Material Icons included and used correctly
44	Styling	Colours applied according to specification
Validation, Security & Deployment		
45	Validation	All HTML pages validate via W3C Validator
46	Validation	CSS validates via W3C CSS Validator
47	Security	.htaccess file is used to prevent unauthorised access
48	Security	All DB queries use prepared statements
49	Security	assets/images/pets/ added to .gitignore
50	Deployment	Site deployed to Coreteaching server, data to Jacob5.
51	Deployment	Permissions set: 755 on a2, 777 on assets/images/pets
52	Deployment	All pages load with no errors on Coreteaching
GitHub & Submission		
53	Git	Project located in a2 directory of GitHub repo
54	Git	Git history shows consistent work over time
55	Git	Teacher added as a collaborator
56	Submission	ZIP named COSC2446_a2_<studentID>.zip
57	Submission	ZIP includes all source files and assets
58	Submission	README.md includes live site URL
Demonstration Requirement		
59	Demo	Able to explain database structure and queries
60	Demo	Able to explain PHP logic and prepared statements
61	Demo	Able to explain JavaScript functionality
62	Demo	Able to explain all submitted code clearly

Submission requirements

You are required to submit your completed project as follows:

COS2446 Web Programming

Deployment

- The live site must be deployed to a **Coreteaching web server** – Jupiter/Saturn/Titan (jupiter.csit.rmit.edu.au/saturn.csit.rmit.edu.au/titan.csit.rmit.edu.au).
- The database for the live deployment must be used on **Jacob 5**, named after your student ID.
- Ensure all links, assets, and dynamic content function correctly on the deployed site.

Canvas Submission

Submit the following as a zipped folder named COSC2446_a2_<yourStudentID>.zip via **Canvas** under the “AT2: Dynamic Website & Demonstration Assignment” assignment:

- All source files and assets.
- A README.md file containing:
 - The **URL of the live site**.

Academic integrity

It is important that you submit material which is your own work. The University provides information for students on academic Integrity through the [RMIT website](#). If you are unsure about academic matters please be sure to ask your teacher for information.

Assessment Policy

Assessments submitted after the due date will be accepted as follows:

- Assessments submitted between 0-24 hours after the due date: 20% penalty
- Assessments submitted between 24-48 hours after the due date: 50% penalty
- Assessments submitted more than 48 hours after the due date will not be accepted

Extensions

If you will not be able to meet the due date for an assessment you may apply to your teacher for an extension of up to seven days by completing the Application of Time to Submit Assessment Work Form at https://www.rmit.edu.au/content/dam/rmit/documents/Students/Student_forms/Application-for-extension-of-time-to-submit-work.pdf Applications for an extension of time must be received at least one working day before the due date for an assessment.

Special Consideration

If unforeseen circumstances beyond your control prevent you from submitting your work on time, you may be eligible to apply for special consideration. For further information regarding special consideration, please refer to the RMIT Special Consideration page at <https://www.rmit.edu.au/students/student-essentials/assessment-and-results/special-consideration>

COSC2446 Web Programming

Functional Requirements

Code and Git Usage

- All code should reside in a “wp” GitHub repository that is under your RMIT student ID username
- The original clone is from <https://github.com/tanyarmit/wp.git>. Add your teacher as a collaborator.
- Work in the a2 directory.
- Use meaningful commit messages and push frequently.
- README.md file should contain the link to your Coreteaching deployment.
- Refer to “Developing and deploying with Git” for instructions on uploading and testing files on Coreteaching web servers.
- Make sure that you follow the naming convention for files and directories. Failure to do so will result in lost marks.
- Students must demonstrate consistent use of Git throughout the project timeline. A minimum of 5 commits spread over at least 4 separate days is expected. No single day can contain more than 50% of the work.

Penalty:

- If there are fewer than 5 commits, or all or 50% of the commits are made on the same day, a 50% penalty will be applied to the overall marks.
- This is to encourage regular progress, documentation of development steps, and authentic authorship.

Page structure & Design

- Match the provided screenshots exactly
- Use semantic HTML5: <header>, <nav>, <main>, <footer>
- Consistent navbar with links, PetConnect_logo.png that is a link to the Home page
- Footer includes a copyright with your name
- Responsive design using Bootstrap 5 for all pages
- Validate all HTML and CSS - <https://validator.w3.org/> and <https://jigsaw.w3.org/css-validator/>.

File structure in a2 folder

```

├── assets/                # Static assets (images, CSS, JS)
│   ├── images/
│   │   └── pets/         # Pet photos
│   ├── css/
│   │   └── styles.css    # CSS
│   └── js/
│       └── scripts.js    # JavaScript
├── includes/             # Include files (header, footer, nav, db connection)
│   ├── db_connect.inc
│   ├── header.inc
│   ├── footer.inc
│   └── nav.inc
└── index.php             # Homepage with carousel and pet listings

```

COSC2446 Web Programming

```

|— add.php           # Add new pet form
|— details.php       # Pet details page
|— gallery.php       # Gallery view of all pets
|— pets.php          # Browse pets by species
|— process_add.php   # Add pet handler. Not mandatory. Adding a new record can
|                   # be handled by add.php

```

Database & Connection

- To create a `petconnect` database on localhost, use the SQL schema provided on Canvas. For Jacob 5, the database name should be your student ID. The database on Jacob 5 will be created for you by RMIT IT support.

Using different tables or column names will lead to a fail mark for the assignment.

- All database access should be performed via **MySQLi procedural** and **prepared statements**.
- Single connection in `includes/db_connect.inc`, with error reporting for development. The file should cater for access to either `localhost` (when run via XAMPP) or Jacob 5 - `talsprddb02.int.its.rmit.edu.au`

Common Includes & Assets

- includes/ folder:**
 - `header.inc`: Bootstrap & custom CSS, Material Icons, favicon links
 - `nav.inc`: navigation links
 - `footer.inc`: gradient background, copyright with your name
 - `db_connect.inc`: handles database connections to localhost and Jacob5
 - Helpers (optional): `functions.inc`, etc.
- Fonts:** Use Google fonts.
 - Headings:** Poppins - <https://fonts.google.com/specimen/Poppins>
 - Weights: 400, 500, 600, 700, 800
 - Usage: All heading elements (h1-h6), navbar brand
 - Body Text:** Inter - <https://fonts.google.com/specimen/Inter>
 - Weights: 400, 500, 600, 700
 - Usage: Body copy, form labels, buttons, navigation links
 - Icons:** Material Icons - <https://fonts.google.com/icons>
- CSS:** There should be one common custom CSS file for all pages - `assets/css/styles.css`. You must create a CSS sheet that will be linked to (in the `<head>` element). This CSS file will contain custom theme colours.
- JS:** There should be one common custom JS file for all pages - `assets/js/scripts.js`. JavaScript should be used for file extension validation and modals. You will need to write JavaScript functions in a separate JavaScript file, linked to the relevant pages. You should link the JS file in the `<head>` or at the end of the `<body>` to ensure best practices.

COSC2446 Web Programming

- **Images:** Images like the logo, banner and favicon should reside in `assets/images`. Pets-related images should reside in `assets/images/pets`. The required images are supplied on Canvas.
- **Colours:** The following colours are used for the site:

Brand Colors

- Primary Color: #6366f1 (Indigo)
- Primary Dark: #4f46e5 (Deep Indigo)
- Primary Light: #818cf8 (Light Indigo)
- Secondary Color: #ec4899 (Pink)
- Secondary Dark: #db2777 (Deep Pink)

Accent Colors

- Accent Orange: #f59e0b
- Accent Green: #10b981
- Accent Purple: #8b5cf6

Neutral Colors

- Text Dark: #1f2937 (Dark Gray)
- Text Light: #6b7280 (Medium Gray)
- Background Light: #f9fafb (Off White)
- Background White: #ffffff (Pure White)
- Border Color: #e5e7eb (Light Gray)

Pages & Features

File	Requirements
index.php	- The same look and feel as in Assignment 1 - Bootstrap carousel of the 4 static images - Responsive grid of 4 latest pets, selected from the database
add.php + processing	- Functional form that results in adding a record to the database and a file to <code>assets/images/pets</code> folder - Form & processing data - Server-side checks, record creation and image preview and upload - Uploaded image files must use unique filenames to prevent overwriting existing files. Acceptable approaches include using the PHP <code>uniqid()</code> function or prefixing or suffixing the original filename with a timestamp (e.g., <code>time()</code> or <code>date()</code> functions) - ID for the database record should be auto generated
pets.php	- Two-column layout: left - banner image, right - table - Table with records from the database - Custom link colours - Name is a hyperlink to <code>details.php</code>, which displays selected record
gallery.php	- Grid of all pet images - Names of the images come from the database. The actual files are in <code>assets/images/pets</code> - Images open in a Bootstrap modal on click - Filters pets based on status via JavaScript

COSC2446 Web Programming

details.php	- Detail page with image. Data comes from the database
--------------------	--

Client-Side JavaScript

- **Image-extension validation** on for the `<input type="file" name="image">` field: only jpg, jpeg, png, gif, webp.
- **Image preview** before the selected file is uploaded on the **add.php** page.
- **Modals** for full-size images **gallery.php** (no inline JS—use `assets/js/scripts.js`).
- **Category filter** on **gallery.php**: pure JavaScript show/hide. Use `data-status` field.

Security & Best Practices

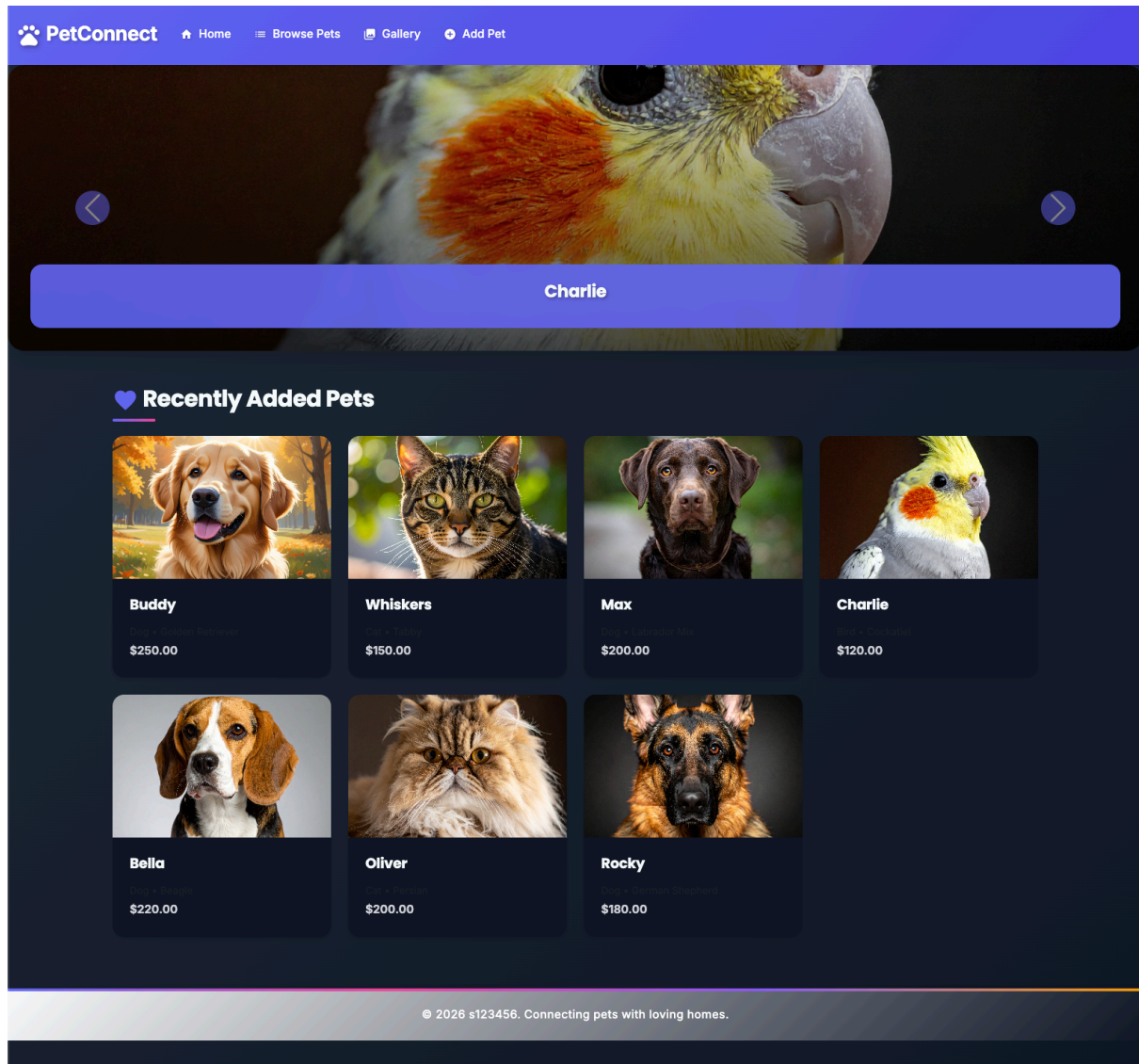
- Prepared statements everywhere. All scripts that use queries with user input, i.e. inserting records, displaying records via detail pages, must be secure and use SQL injection prevention methods – prepared statements.
- Maintain clean uploads (e.g. unique file names). You must not rely on the original uploaded filename. A unique name must be generated server-side using methods such as `uniqid()` or a timestamp-based naming convention.
- Add `assets/images/pets/` to `.gitignore` to prevent sync issues.

Submission Instructions

1. Push to your private Git repository with clear commits.
2. Deploy to the Coreteaching web servers and ensure all assets load correctly.
3. Use the `.htaccess` file to prevent unauthorised access to your site on Coreteaching servers.
4. Submit via Canvas a zipped folder with:
 - All files
 - Live site URL in `README.md`

NOTE: The following screenshots are provided as reference for the required look and feel. Students are expected to replicate these **exactly** in their implementations.

Home page:



Pets page:

COSC2446 Web Programming

PetConnect Home Browse Pets Gallery Add Pet

All Available Pets



NAME	SPECIES	BREED	SIZE	FEE (\$)
Buddy	Dog	Golden Retriever	Large	250.00
Whiskers	Cat	Tabby	Small	150.00
Max	Dog	Labrador Mix	Large	200.00
Charlie	Bird	Cockatiel	Small	120.00
Bella	Dog	Beagle	Medium	220.00
Oliver	Cat	Persian	Medium	200.00
Rocky	Dog	German Shepherd	Extra Large	180.00

© 2026 s123456. Connecting pets with loving homes.

Gallery:

PetConnect Home Browse Pets Gallery Add Pet

Pet Gallery

Filter by Status: Show All



Buddy
Dog Available
\$250.00



Whiskers
Cat Available
\$150.00



Max
Dog Available
\$200.00



Luna
Cat Pending
\$180.00



Charlie
Bird Available
\$120.00



Bella
Dog Available
\$220.00



Oliver
Cat Available
\$200.00

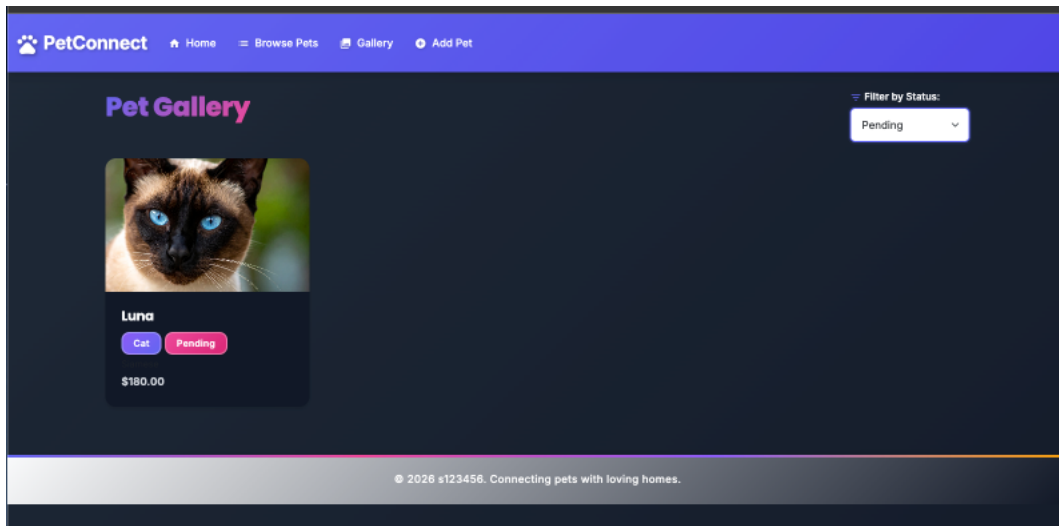


Rocky
Dog Available
\$180.00

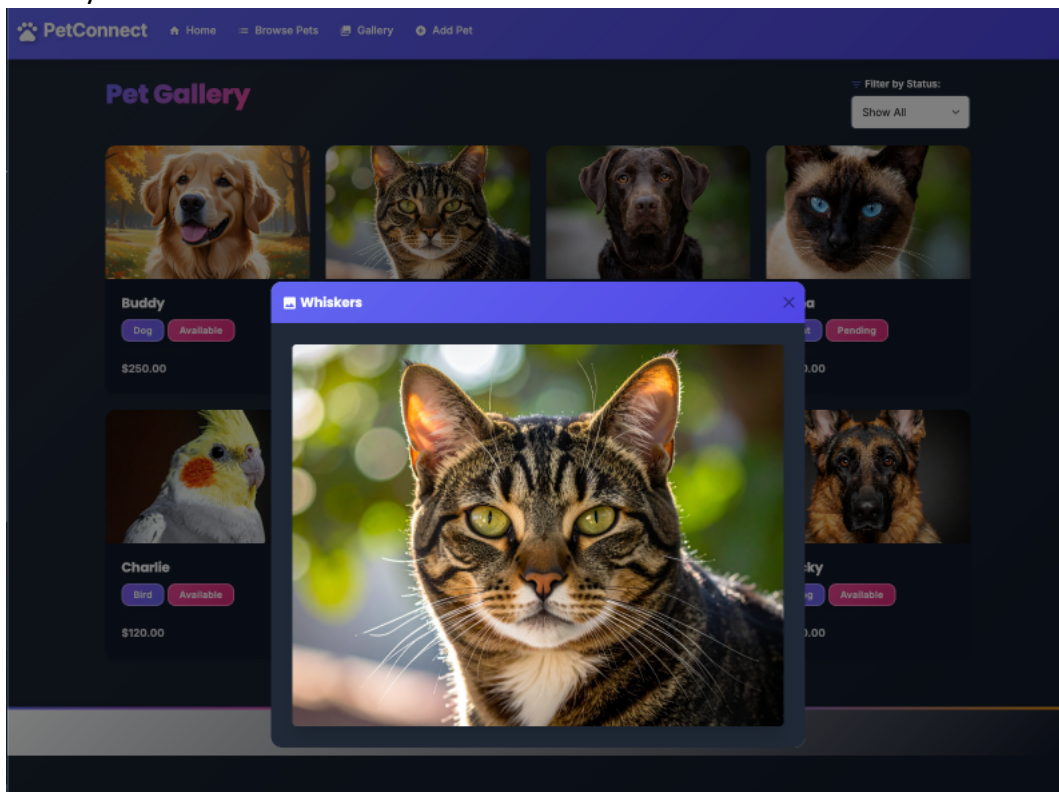
© 2026 s123456. Connecting pets with loving homes.

Gallery with filter:

COSC2446 Web Programming



Gallery with modal:



Add pet:

 **PetConnect** [Home](#) [Browse Pets](#) [Gallery](#) [Add Pet](#)

Add a New Pet for Adoption

Pet Name *

Species *

Breed

Age (Years)

Age (Months)

Gender *

Size *

Adoption Fee (\$) *

Description *

Describe the pet's personality and characteristics

Health Information

Enter health and vaccination information

Status *

Pet Photo

© 2026 s123456. Connecting pets with loving homes.

COSC2446 Web Programming

Add pet with image preview:

PetConnect Home Browse Pets Gallery Add Pet

Add a New Pet for Adoption

Pet Name * **Species ***

Breed **Age (Years)** **Age (Months)**

Gender * **Size *** **Adoption Fee (\$) ***

Description *
A lovely miniature poodle, great with adults and kids

Health Information
Poor eyesight

Status *
Available

Pet Photo
Choose file

Valid image selected: Roxi.jpg (4791.69 KB)



© 2026 s123456. Connecting pets with loving homes.

Details page:

PetConnect Home Browse Pets Gallery Add Pet

Buddy

Breed:	Golden Retriever
Age:	3 years, 6 months
Gender:	Male
Size:	Large
Adoption Fee:	\$250.00

Description
Friendly and energetic golden retriever looking for an active family. Loves playing fetch and swimming.

Health Information
Fully vaccinated, microchipped, neutered. No known health issues.

© 2026 s123456. Connecting pets with loving homes.